

---

---

# Air Quality Prediction System

---

---

## Project Id: 19

Name : Rounak Tayal (Team Leader)

Class Roll Number:56

University Roll Number : 2028826

Section and Group: 'U' and G2

Name : Vansh Choudhary

Class Roll Number: 65

University Roll Number : 2028898

Section and Group: 'U' and G2

Name : Keshav Sharma

Class Roll Number: 45

University Roll Number : 2028741

Section and Group: 'U' and G2

## TABLE OF CONTENTS

|                                                   |                |
|---------------------------------------------------|----------------|
| <b>1. Problem Description .....</b>               | <b>3</b>       |
| <b>1.1. Dataset Description .....</b>             | <b>3</b>       |
| <b>1.2. Flow diagram.....</b>                     | <b>4</b>       |
| <b>2. Modules for Project Implementation.....</b> | <b>5</b>       |
| <b>2.1. Modules Used .....</b>                    | <b>5 – 6</b>   |
| <b>2.2. Platform Used .....</b>                   | <b>7</b>       |
| <b>3. Machine Learning model used .....</b>       | <b>8 – 9</b>   |
| <b>4. Evaluation metrics used .....</b>           | <b>10 – 11</b> |
| <b>5. Results and Visualizations .....</b>        | <b>12 – 15</b> |
| <b>6. Conclusion and Future Scope .....</b>       | <b>16</b>      |
| <b>7. References.....</b>                         | <b>17</b>      |

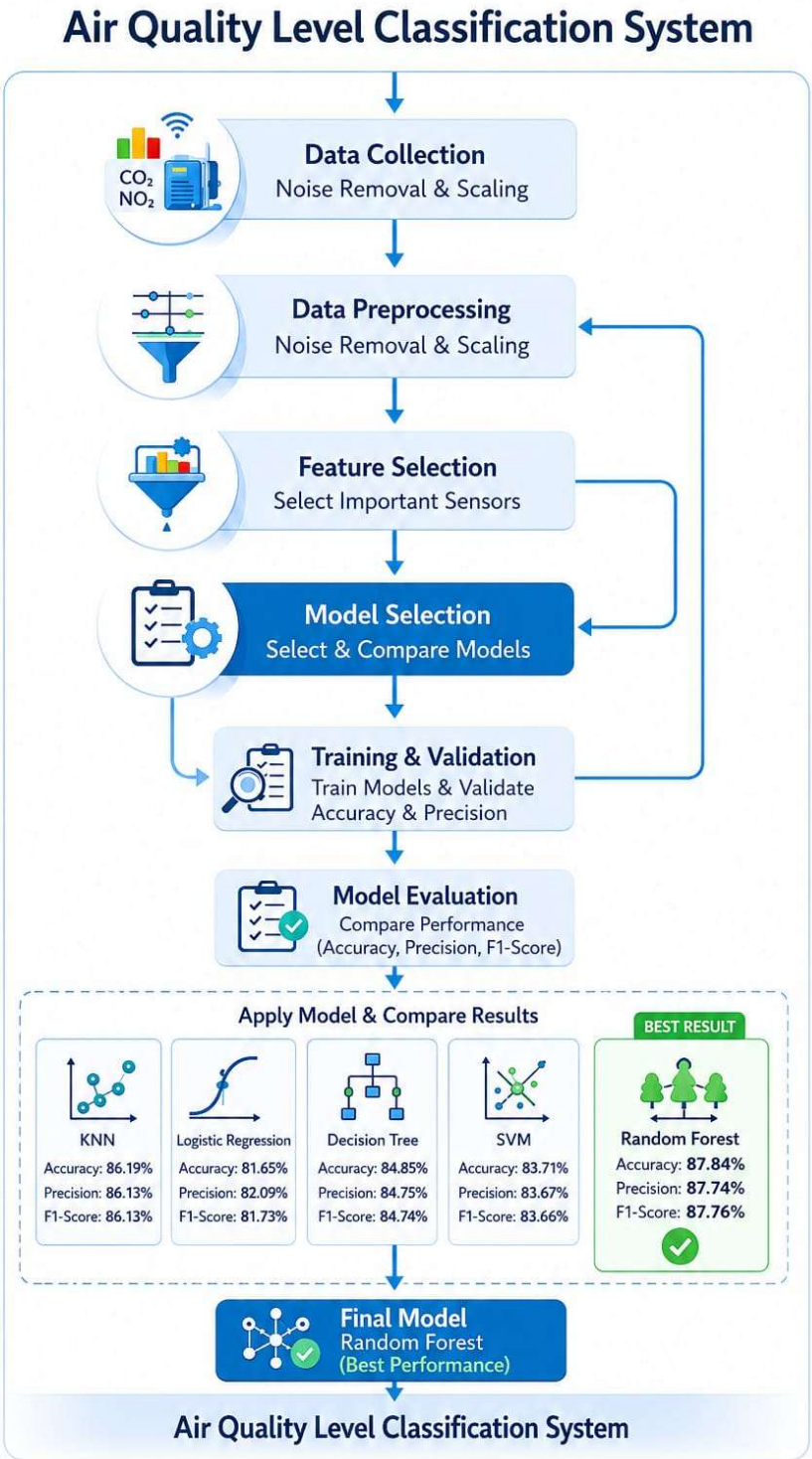
## **1. Problem Description**

Air pollution is a growing environmental concern that directly affects public health and quality of life. Monitoring air quality in real time and classifying it into meaningful health categories is essential for timely decision-making by governments and the general public. This project aims to automate the classification of Air Quality Index (AQI) readings into three categories — GOOD, Moderate, and WORST — using machine learning algorithms applied to sensor-collected pollutant data.

### **1.1 Dataset Description**

The dataset consists of 5000 records with five attributes: Carbon Monoxide (CO), Nitrogen Dioxide (NO<sub>2</sub>), Temperature, Humidity, and Air Quality Index. CO and NO<sub>2</sub> are the primary pollutants and act as the main drivers of the AQI value. The dataset was designed to reflect realistic sensor conditions and was intentionally injected with approximately 5% missing values per column, 2% outliers, and 1% duplicate records to simulate real-world data quality challenges.

## 1.2 Flow Diagram



## 2. Modules for Project Implementation

**1. Pandas** Pandas has been used for data manipulation, analysis, and handling structured tabular data throughout the project.

Functions used:

- `pd.read_csv()` — to read the air quality dataset from a CSV file into a DataFrame
- `df.head()` — to display the first few rows of the dataset
- `df.info()` — to get a summary of the dataset including column types and null counts
- `df.describe()` — to get statistical summary of all numerical columns
- `df.isnull().sum()` — to count missing values in each column
- `df.drop_duplicates()` — to remove duplicate rows from the dataset
- `df.dropna()` — to drop rows where the target column has null values
- `df.copy()` — to create an independent copy of the DataFrame during outlier handling
- `df.corr()` — to compute pairwise correlation between all feature columns
- `df['col'].quantile()` — to calculate Q1 and Q3 for IQR-based outlier detection
- `df['col'].clip()` — to clip outlier values within the IQR bounds
- `df['col'].value_counts()` — to check class distribution of the target column
- `df['col'].apply()` — to apply the labeling function to convert AQI values to categories

**2. Matplotlib** Matplotlib has been used for creating static visualizations and plots throughout the project.

Functions used:

- `plt.figure()` — to create a new figure for plotting
- `plt.bar()` — to plot bar charts for class distribution and model comparison
- `plt.show()` — to display the generated plots
- `plt.title()` — to set the title of a plot
- `plt.xlabel()` / `plt.ylabel()` — to label the axes
- `plt.xticks()` — to rotate x-axis tick labels for better readability
- `plt.tight_layout()` — to automatically adjust subplot spacing
- `plt.text()` — to annotate accuracy values on top of bars in comparison chart
- `plt.subplots()` — to create multiple subplots side by side for confusion matrices

**3. Seaborn** Seaborn has been used for creating enhanced statistical visualizations built on top of Matplotlib.

Functions used:

- `sns.boxplot()` — to visualize feature distributions and detect outliers before and after cleaning

**4. Warnings** The warnings module has been used to suppress unnecessary warning messages during model training and evaluation.

Functions used:

- `warnings.filterwarnings("ignore")` — to suppress all runtime and convergence warnings

## **5. Scikit-learn — Model Selection** from sklearn.model\_selection

Functions used:

- train\_test\_split() — to split the dataset into 80% training and 20% testing sets
- GridSearchCV() — to perform exhaustive hyperparameter tuning with cross-validation for all models
- cross\_val\_score() — to evaluate model performance using k-fold cross-validation

## **6. Scikit-learn — Preprocessing** from sklearn.preprocessing

Functions used:

- StandardScaler() — to standardize feature values by removing mean and scaling to unit variance; fitted only on training data and applied to both train and test sets

## **7. Scikit-learn — Feature Selection** from sklearn.feature\_selection

Functions used:

- SequentialFeatureSelector() — to perform Forward and Backward feature selection to identify the top 3 most relevant features for distance-based models (KNN, LR, SVM)

## **8. Scikit-learn — Classification Models** from sklearn.neighbors, sklearn.linear\_model, sklearn.tree, sklearn.ensemble, sklearn.svm

Models used:

- KNeighborsClassifier() — K-Nearest Neighbors classifier
- LogisticRegression() — Logistic Regression classifier with multinomial support
- DecisionTreeClassifier() — Decision Tree classifier with Gini and Entropy criteria
- RandomForestClassifier() — Random Forest ensemble classifier
- SVC() — Support Vector Machine classifier with RBF and Polynomial kernels

## **9. Scikit-learn — Evaluation Metrics** from sklearn.metrics

Functions used:

- accuracy\_score() — to compute the overall classification accuracy
- precision\_score() — to compute weighted precision across all three classes
- recall\_score() — to compute weighted recall across all three classes
- f1\_score() — to compute both weighted and macro F1-Score
- classification\_report() — to generate a detailed per-class evaluation report
- confusion\_matrix() — to generate the confusion matrix for visualizing prediction errors

**10. Pickle** (*Custom Save/Load*) Pickle is a Python built-in module used for serializing and deserializing Python objects. In this project it was used to save the trained Random Forest model and the StandardScaler object to disk so they can be reloaded later for prediction without retraining.

Functions used:

- pickle.dump() — to serialize and save the model and scaler into a .pkl file
- pickle.load() — to load the saved model and scaler back into memory in the prediction script

## 2.2 Platform Used:

- **Jupyter Notebook:** Jupyter Notebook was the only platform used throughout the entire development of this project. It provides an interactive cell-based environment where code, output, and visualizations can be viewed together in a single interface, making it ideal for data science and machine learning workflows. Each step of the project — data generation, preprocessing, feature selection, model training, hyperparameter tuning, and evaluation — was implemented and executed in separate cells within Jupyter Notebook, allowing easy debugging and step-by-step verification of results. All plots including boxplots, correlation heatmaps, confusion matrices, and model comparison charts were rendered inline within the notebook itself. The final trained Random Forest model and StandardScaler were saved as a .pkl file directly from a Jupyter Notebook cell using the Pickle module. The standalone Python prediction script was also written and tested within Jupyter Notebook and later executed from the terminal for real-time air quality prediction, making Jupyter Notebook the single unified platform for both model development and deployment in this project.

### 3. Machine Learning Model Used :

#### 1. K-Nearest Neighbors (KNN)

K-Nearest Neighbors is a non-parametric, instance-based supervised learning algorithm that classifies a new data point by looking at the K closest data points in the training set and assigning the majority class among them. It does not build an explicit model during training — instead it memorizes the entire training dataset and performs computation only at the time of prediction, which is why it is called a lazy learner. The distance between data points is measured using Euclidean or Manhattan distance metrics. In this project, KNN was trained on the 3 forward-selected features and tuned using GridSearchCV with K values of 3, 5, 7, 9, and 11, along with uniform and distance-based weighting strategies. KNN achieved an accuracy of **86.19%** on the air quality dataset.

#### 2. Logistic Regression

Logistic Regression is a linear supervised classification algorithm that models the probability of a data point belonging to a particular class using the logistic (sigmoid) function. Despite the word regression in its name, it is fundamentally a classification algorithm that finds the optimal decision boundary separating the classes. For multi-class problems like this project, it uses the multinomial approach where probabilities are computed across all classes simultaneously. Regularization parameter C controls the trade-off between fitting the training data and keeping the model simple to avoid overfitting. In this project, Logistic Regression was trained on the 3 forward-selected features using lbfgs and saga solvers, and the regularization strength was tuned using GridSearchCV. Logistic Regression achieved an accuracy of **81.65%** on the air quality dataset.

#### 3. Decision Tree

A Decision Tree is a non-linear supervised learning algorithm that splits the dataset into subsets based on the most informative feature at each node, forming a tree-like structure of decisions. At each internal node, the algorithm selects the feature and threshold that best separates the classes according to a splitting criterion — either Gini Impurity or Information Gain (Entropy). The tree continues splitting until it reaches a leaf node that represents a final class prediction. Decision Trees are highly interpretable as the learned rules can be visualized and understood easily. However they are prone to overfitting, which is controlled by limiting the maximum depth of the tree. In this project, Decision Tree was trained on all 4 scaled features and tuned using GridSearchCV with varying max depth values of 3, 5, 7, 10 and None, minimum samples split values, and both Gini and Entropy criteria. Decision Tree achieved an accuracy of **84.85%** on the air quality dataset.

#### 4. Random Forest

Random Forest is a powerful ensemble learning algorithm that builds multiple Decision Trees during training and combines their predictions through majority voting to produce the final classification. Each tree in the forest is trained on a random bootstrap sample of the training data, and at each node only a random subset of features is considered for splitting — a technique called feature randomness. This combination of data randomness and feature randomness makes Random Forest highly robust to noise, outliers, and overfitting compared to a single Decision Tree. It also provides feature importance scores that indicate how much each feature contributed to the predictions. In this project, Random Forest was trained on all 4 scaled features and tuned using GridSearchCV with 100 and 200 estimators, varying max depth values, and sqrt and log2 feature selection strategies. Random Forest achieved the highest accuracy of **87.84%** among all models and was selected as the final deployed model for the air quality prediction system.

#### 5. Support Vector Machine (SVM)

Support Vector Machine is a powerful supervised classification algorithm that works by finding the optimal hyperplane that maximally separates the classes in the feature space. The data points closest to the decision boundary are called support vectors, and SVM tries to maximize the margin between these support vectors and the hyperplane. For non-linearly separable data, SVM uses the kernel trick to transform the data into a higher-dimensional space where a linear separation becomes possible. The RBF (Radial Basis Function) kernel and Polynomial kernel are commonly used for this transformation. The regularization parameter C controls the trade-off between maximizing the margin and minimizing classification errors, while the gamma parameter controls the influence of individual training points on the decision boundary. In this project, SVM was trained on the 3 forward-selected features and tuned using GridSearchCV with C values of 0.1, 1, 10 and 100, RBF and Polynomial kernels, and scale and auto gamma settings. SVM achieved an accuracy of **83.71%** on the air quality dataset.

## 4. Evaluation metrics used :

### 1. Accuracy

Accuracy is the most fundamental evaluation metric that measures the ratio of correctly classified samples to the total number of samples in the dataset. It provides a quick overall view of how well the model is performing across all three air quality classes. In this project accuracy was the primary metric used to rank and compare all five models in the final evaluation summary table.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

### 2. Precision

Precision measures the proportion of samples that were correctly predicted as belonging to a particular class out of all samples that the model predicted as that class. It answers the question — out of all the times the model predicted WORST air quality, how many times was it actually WORST. High precision means the model makes very few false positive errors. In this project weighted precision was calculated across all three classes — GOOD, Moderate, and WORST — using the classification report.

$$Precision = \frac{TP}{TP + FP}$$

### 3. Recall

Recall measures the proportion of actual samples belonging to a class that were correctly identified by the model. It answers the question — out of all the actual WORST air quality cases, how many did the model successfully detect. High recall is especially critical for the WORST class in this project because failing to detect severely polluted air conditions could have serious public health consequences. Weighted recall was reported for all models using the classification report.

$$Recall = \frac{TP}{TP + FN}$$

### 4. F1-Score

F1-Score is the harmonic mean of Precision and Recall and provides a single balanced metric that takes both false positives and false negatives into account. It is more informative than accuracy alone when the class distribution is not perfectly balanced. In this project the classification report generated both per-class F1-Score for GOOD, Moderate, and WORST individually as well as the overall weighted average F1-Score for each model.

$$F1-Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

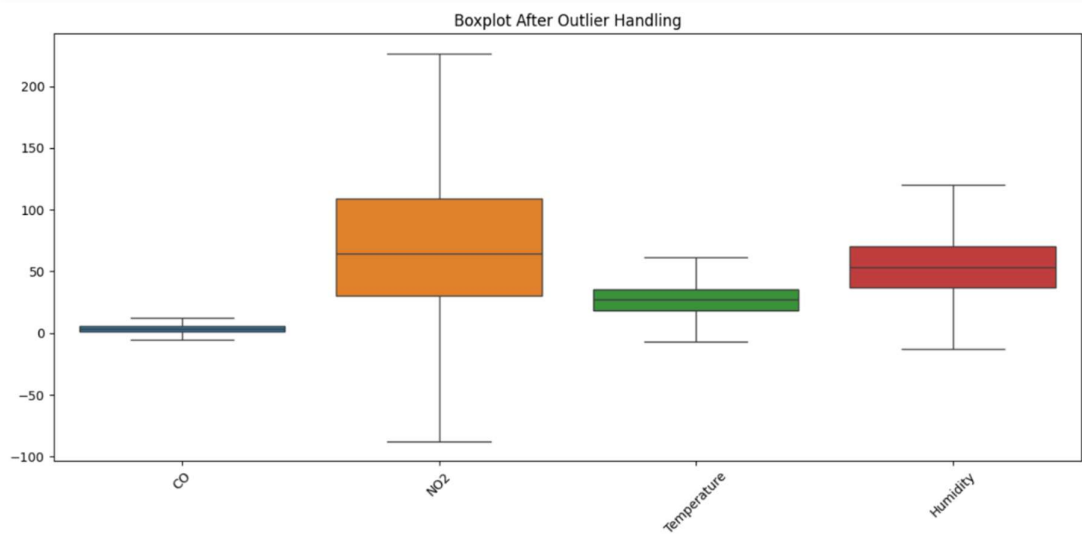
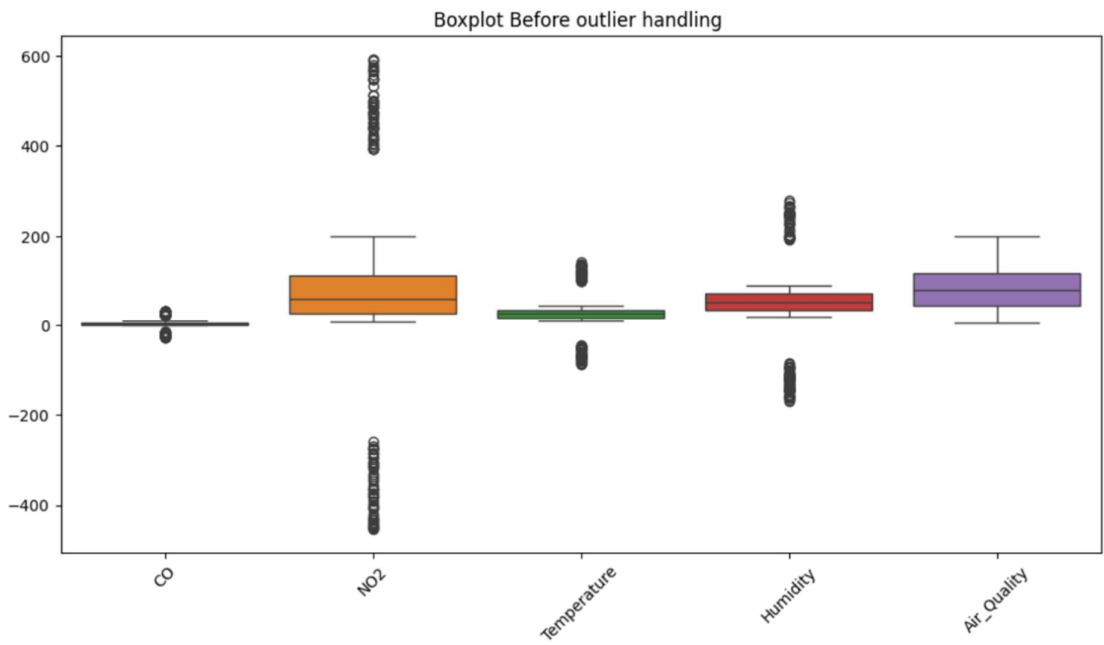
## **5. Classification Report**

The Classification Report is a consolidated evaluation summary generated using `classification_report()` from scikit-learn. It combines Precision, Recall, and F1-Score for each individual class into a single structured output along with support values indicating the number of actual samples per class. It also provides macro average which treats all classes equally regardless of size and weighted average which accounts for class size. In this project a classification report was printed for all five models — KNN, Logistic Regression, Decision Tree, Random Forest, and SVM — giving a complete per-class breakdown of performance across GOOD, Moderate, and WORST categories.

## **6. Model Evaluation Summary Table**

A consolidated Model Evaluation Summary Table was generated at the end of the notebook comparing all five models side by side on Accuracy, Precision, Recall, and F1-Score in a single pandas DataFrame. The models were sorted in descending order of accuracy to clearly identify the best performing model. This summary served as the final comparative analysis of all trained models in the project

## 5. Results and Visualizations



KNN Accuracy: 0.8619 (86.19%)

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| GOOD         | 0.88      | 0.93   | 0.90     | 332     |
| Moderate     | 0.81      | 0.78   | 0.79     | 319     |
| WORST        | 0.90      | 0.87   | 0.88     | 319     |
| accuracy     |           |        | 0.86     | 970     |
| macro avg    | 0.86      | 0.86   | 0.86     | 970     |
| weighted avg | 0.86      | 0.86   | 0.86     | 970     |

Logistic Regression Accuracy: 0.8165 (81.65%)

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| GOOD         | 0.83      | 0.88   | 0.85     | 332     |
| Moderate     | 0.73      | 0.76   | 0.74     | 319     |
| WORST        | 0.91      | 0.81   | 0.86     | 319     |
| accuracy     |           |        | 0.82     | 970     |
| macro avg    | 0.82      | 0.82   | 0.82     | 970     |
| weighted avg | 0.82      | 0.82   | 0.82     | 970     |

Decision Tree Accuracy: 84.85%

Decision Tree

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| GOOD         | 0.88      | 0.94   | 0.91     | 332     |
| Moderate     | 0.79      | 0.76   | 0.78     | 319     |
| WORST        | 0.88      | 0.84   | 0.86     | 319     |
| accuracy     |           |        | 0.85     | 970     |
| macro avg    | 0.85      | 0.85   | 0.85     | 970     |
| weighted avg | 0.85      | 0.85   | 0.85     | 970     |

Random Forest Accuracy: 87.84%

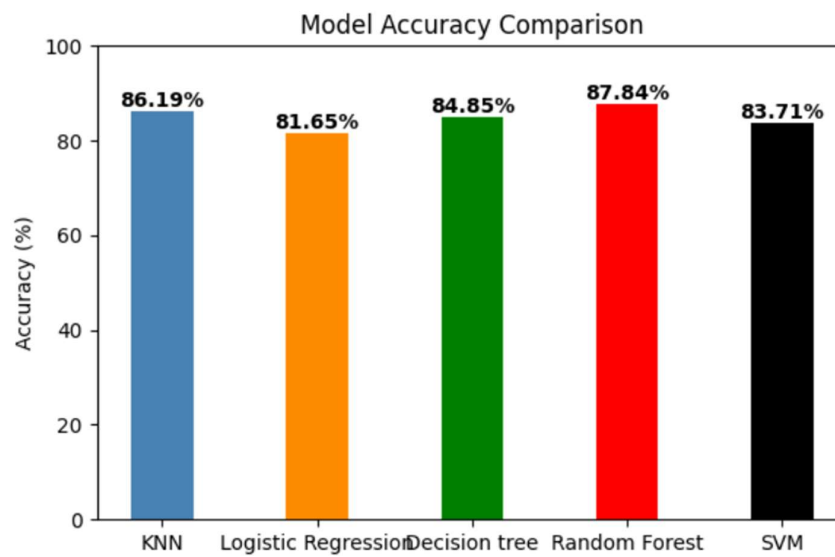
Random Forest

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| GOOD         | 0.90      | 0.93   | 0.91     | 332     |
| Moderate     | 0.84      | 0.80   | 0.82     | 319     |
| WORST        | 0.90      | 0.91   | 0.90     | 319     |
| accuracy     |           |        | 0.88     | 970     |
| macro avg    | 0.88      | 0.88   | 0.88     | 970     |
| weighted avg | 0.88      | 0.88   | 0.88     | 970     |

SVM Accuracy: 83.71%

SVM

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| GOOD         | 0.84      | 0.89   | 0.87     | 332     |
| Moderate     | 0.76      | 0.74   | 0.75     | 319     |
| WORST        | 0.90      | 0.88   | 0.89     | 319     |
| accuracy     |           |        | 0.84     | 970     |
| macro avg    | 0.84      | 0.84   | 0.84     | 970     |
| weighted avg | 0.84      | 0.84   | 0.84     | 970     |



Best Model: Random Forest with 87.84% accuracy

## 6. Conclusion and Future Scope:

In this project, a machine learning-based approach was used to classify air quality levels using environmental sensor data. The dataset included parameters such as CO, NO<sub>2</sub>, temperature, and humidity. Before applying any models, the data was carefully preprocessed to handle missing values, remove noise, and bring all features to a similar scale. This step was important to ensure that the models could learn effectively from the data.

Feature selection was also performed to focus only on the most important attributes. This not only reduced unnecessary complexity but also helped in improving the overall performance of the models.

To understand which approach works best, multiple algorithms were tested, including KNN, Logistic Regression, SVM, Decision Tree, and Random Forest. It was observed that each model behaved differently depending on the data. Tree-based methods, especially Random Forest, generally gave more stable and accurate results because they can handle complex and non-linear patterns well. On the other hand, simpler models like Logistic Regression and KNN worked well in certain cases but were more affected by how the data was distributed and scaled. SVM also performed well but needed careful tuning of parameters to achieve good results.

Overall, this project shows that building a reliable air quality classification system is not just about choosing one model, but about properly preparing the data and comparing different approaches to find what works best.

Looking ahead, there is a lot of scope to improve this system. Using a larger and more diverse dataset can make the model more robust and reliable. The project can also be extended to work with real-time sensor data, which would make it useful for live air quality monitoring. Further improvements can be made by tuning model parameters and trying more advanced techniques like ensemble methods or deep learning.

In addition, including factors such as weather conditions, traffic levels, and location data can make the predictions more accurate and practical. From a usability point of view, developing a simple dashboard or mobile app can help users easily understand air quality levels and get real-time updates. In the future, such systems can also be integrated into smart city solutions to support better environmental monitoring and decision-making.

## 6. References:

### 1. Books:

1. Let Us Python – Yashavant Kanetkar
2. Scikit learn Documentation

### 2. Online Resources:

1. GeeksforGeeks – <https://www.geeksforgeeks.org/>
2. Kaggle – <https://www.kaggle.com>